How to Apply the Multi-Project Upgrade

Step-by-step for non-technical users

Before You Start

Make sure CEREBRO is already running locally. If you haven't set it up yet, follow the CEREBRO_Setup_Guide.md first.

You should have:

- A `cerebro` folder with the Next.js project
- The app running at http://localhost:3000 (or you know how to start it with `npm run dev`)

Step 1: Download the change brief

Download `CEREBRO_Change_Brief_MultiProject.md` from this conversation.

Put it in your `cerebro` project folder — the same folder where your code lives. You can drag it into the VS Code file explorer on the left side.

Your folder should now look something like:

```
cerebro/
  ├── CEREBRO_Change_Brief_MultiProject.md   ← the new file
  ├── app/
  ├── package.json
  ├── ... (other project files)
```

Step 2: Open Claude Code

In VS Code, open the Claude Code panel (sidebar icon or terminal → type `claude`).

Step 3: Paste this prompt into Claude Code

Copy this EXACT prompt and paste it:

```
Read the file CEREBRO_Change_Brief_MultiProject.md in this project folder.

This describes an upgrade from single-project to multi-project support.
Follow the instructions in the change brief exactly.

Create an update plan first — list every file you'll change, what you'll
change in it, and the order of operations.

Do NOT write any code yet. Show me the plan and wait for my approval.
```

Step 4: Review the plan

Claude Code will read the change brief and present a plan like:

```
Plan:
1. Update the data layer — change localStorage from flat to projects array
2. Create ProjectDashboard component — card grid, new project button
3. Update the main App component — add project switching, nav changes
4. Update state management — load/save per project
5. Add migration logic for existing data
6. Test all flows
```

Read the plan carefully. If anything looks wrong or missing, tell Claude Code. For example:

- "The plan doesn't mention the test data button — make sure that still works"
- "Add project status badges to the plan"

When you're happy with the plan, type:

```
Plan approved. Start implementing.
```

Step 5: Let Claude Code work

Claude Code will make the changes file by file. This usually takes 5-10 minutes.

Don't interrupt it unless you see an obvious error. Claude Code will update you as it goes.

Step 6: Test in your browser

Once Claude Code says it's done:

1. If the dev server is still running, it should auto-refresh
2. If not, type `npm run dev` in the terminal
3. Go to http://localhost:3000

You should see the new Project Dashboard. Test:

- Create a new project
- Fill in some answers and generate
- Go back to dashboard
- Create a second project
- Switch between them
- Check that versions work per project

Step 7: If something breaks

If the app crashes or something looks wrong, tell Claude Code:

```
The app is showing an error. Check the terminal and browser console for
errors and fix them.
```

Or be specific:

```
When I click "New Project" nothing happens. Fix this.
```

Claude Code will diagnose and fix the issue.

Step 8: Save your work

Once everything works, type in Claude Code:

```
All looks good. Commit these changes with the message:
"feat: multi-project support with dashboard and project switching"
```

This saves a checkpoint in Git so you can always come back to this version.

Quick Reference: Claude Code Commands

What you want	What to type in Claude Code
Start implementing	`Plan approved. Start implementing.`
Something broke	`The app is broken. Check errors and fix.`
Test it	`Run the dev server so I can test in the browser.`
Save progress	`Commit with message: "description of what changed"`
Undo everything	`Revert all uncommitted changes.`
See what changed	`Show me a summary of all files that changed.`

What This Pattern Looks Like for Future Changes

This is the CEREBRO iteration loop in action:

1. You describe what you want (in a change brief or just in words)
2. Claude Code reads it and makes a plan
3. You approve the plan
4. Claude Code implements it
5. You test in the browser
6. You commit when happy

Every future upgrade — adding API proxy, deploying to Vercel, adding team features — follows this exact same pattern. The change brief just tells Claude Code what to do. You stay in control.